Bayesian Learning

Machine Learning Course - Anca Ralescu

Topics Covered

- Bayes Theorem
- Maximum a posteriori (MAP) hypotheses
- Maximum likelihood (ML) hypotheses
- MAP learners
- Minimum description length principle
- Bayes optimal classifier
- Naive Bayes learner
- Example: learning over text data
- Bayesian belief networks
- Expectation Maximization (EM) algorithm

Outline

1. Bayes Theorem and MAP/ML Hypotheses
2. MAP Learners and Concept Learning
3. ML Hypotheses for Real-Valued and Probabilistic Targets
4. Minimum Description Length Principle
5. Bayes Optimal and Gibbs Classifiers
6. Naive Bayes Classifier
7. Learning to Classify Text
8. Bayesian Belief Networks
9. Expectation Maximization

Two Roles for Bayesian Methods

Practical learning algorithms

- Naive Bayes learning
- Bayesian belief network learning
- Combine **prior knowledge** (prior probabilities) with **observed data**
- Requires prior probabilities

Useful conceptual framework

- Provides a "gold standard" for evaluating other learning algorithms
- Additional insight into Occam's razor
- Lets us characterize algorithms that never mention probabilities explicitly

Bayes Theorem

Idea: under certain conditions, *reverse the arrow* $X \longrightarrow Y$ to infer something about X given knowledge of Y. The arrow is modelled as a conditional probability.

In learning: D is the set of training examples for a concept and h is a candidate hypothesis.

Bayes Theorem

$$P(h \mid D) = \frac{P(D \mid h)\, P(h)}{P(D)}$$

- $P(h)$ — **prior** probability of hypothesis h
- $P(D)$ — prior probability of training data D
- $P(h \mid D)$ — **posterior** probability of h given D
- $P(D \mid h)$ — probability of D given h (the **likelihood**)

Reading Bayes Theorem

$$\underbrace{P(h \mid D)}_{\text{posterior}} = \frac{\overbrace{P(D \mid h)}^{\text{likelihood}} \overbrace{P(h)}^{\text{prior}}}{\underbrace{P(D)}_{\text{evidence}}}$$

- $P(h \mid D)$ *increases* with $P(h)$: a priori plausible hypotheses are favored.
- $P(h \mid D)$ *increases* with $P(D \mid h)$: hypotheses that explain the data well are favored.
- $P(h \mid D)$ *decreases* with $P(D)$: data that are likely regardless of h provide little evidence for h.
- $P(D)$ does not depend on h — it is the same normalizing constant for every hypothesis.

Choosing Hypotheses: the MAP Hypothesis

We usually want the **most probable hypothesis given the training data**: the *maximum a posteriori* hypothesis.

$$
\begin{aligned}
h_{\mathrm{MAP}} &= \arg\max_{h \in H} P(h \mid D) \\
&= \arg\max_{h \in H} \frac{P(D \mid h)\, P(h)}{P(D)} \\
&= \arg\max_{h \in H} P(D \mid h)\, P(h)
\end{aligned} \tag{1}
$$

Why can we drop $P(D)$?

$P(D)$ is a positive constant independent of h, so dividing by it does not change which h achieves the maximum.

Maximum Likelihood (ML) Hypothesis

Assume a **uniform prior** on the hypothesis space:

$$\forall h \in H: \quad P(h) = \frac{1}{|H|}.$$

Then $P(h)$ is also a constant and (1) simplifies to

$$h_{\mathrm{MAP}} = \arg\max_{h \in H} P(D \mid h). \tag{2}$$

Maximum likelihood hypothesis

$P(D \mid h)$ is called the **likelihood** of the data given h; the solution of (2) is

$$h_{\mathrm{ML}} = \arg\max_{h \in H} P(D \mid h).$$

Take-away: under a uniform prior, $h_{\mathrm{MAP}} = h_{\mathrm{ML}}$.

Example 1: Bayes Theorem for Diagnosis

Does the patient have cancer or not?

A patient takes a lab test and the result comes back **positive**.

- The test gives a correct positive result in only 98% of cases where the disease is present.
- It gives a correct negative result in only 97% of cases where the disease is absent.
- 0.008 of the entire population has this cancer.

$P(\text{cancer}) = 0.008$	$P(\neg\text{cancer}) = 0.992$
$P(+ \mid \text{cancer}) = 0.98$	$P(- \mid \text{cancer}) = 0.02$
$P(+ \mid \neg\text{cancer}) = 0.03$	$P(- \mid \neg\text{cancer}) = 0.97$

Example 1: Computing the MAP Hypothesis

Compare $P(+ \mid h)\,P(h)$ for the two hypotheses:

$$P(+ \mid \text{cancer})\,P(\text{cancer}) = 0.98 \times 0.008 = 0.00784$$
$$P(+ \mid \neg\text{cancer})\,P(\neg\text{cancer}) = 0.03 \times 0.992 = 0.02976$$

$$\Rightarrow \quad h_{\text{MAP}} = \neg\text{cancer}$$

Exact posterior (normalize using the total probability $P(+) = 0.00784 + 0.02976 = 0.0376$):

$$P(\text{cancer} \mid +) = \frac{0.00784}{0.0376} \approx 0.21, \qquad P(\neg\text{cancer} \mid +) \approx 0.79.$$

Lesson

The positive result raises the probability of cancer from 0.8% to about 21% — but the low prior still dominates.

Basic Formulas for Probabilities

Product rule

Probability of a conjunction of two events A and B:

$$P(A \wedge B) = P(A \mid B)\, P(B) = P(B \mid A)\, P(A)$$

Sum rule

Probability of a disjunction of two events A and B:

$$P(A \vee B) = P(A) + P(B) - P(A \wedge B)$$

Theorem of total probability

If $A_1, \ldots, A_n$ are mutually exclusive with $\sum_{i=1}^{n} P(A_i) = 1$, then

$$P(B) = \sum_{i=1}^{n} P(B \mid A_i)\, P(A_i)$$

Bayes theorem follows directly from the product rule; total probability gives $P(D)$,
the relation between the joint probability (probability of the intersection and conditional probability.

Outline

1. Bayes Theorem and MAP/ML Hypotheses
2. MAP Learners and Concept Learning
3. ML Hypotheses for Real-Valued and Probabilistic Targets
4. Minimum Description Length Principle
5. Bayes Optimal and Gibbs Classifiers
6. Naive Bayes Classifier
7. Learning to Classify Text
8. Bayesian Belief Networks
9. Expectation Maximization

Brute-Force MAP Hypothesis Learner

Algorithm

1. For each hypothesis $h \in H$, calculate the posterior probability

$$P(h \mid D) = \frac{P(D \mid h)\, P(h)}{P(D)}$$

2. Output the hypothesis with the highest posterior probability

$$h_{\mathrm{MAP}} = \arg\max_{h \in H} P(h \mid D)$$

- Conceptually simple, but requires computing $P(h \mid D)$ for *every* h.
- Impractical for large H — yet extremely useful as a **reference standard** for analysing other learners.

Relation to Concept Learning: Setup

Our usual concept learning task:

- instance space X, hypothesis space H, training examples D
- FIND-S: outputs the most specific hypothesis in the version space $VS_{H,D}$

Questions: What would Bayes rule produce as the MAP hypothesis? Does FIND-S output a MAP hypothesis?

Assumptions:

- Fixed set of instances $\langle x_1, \ldots, x_m \rangle$
- D is the set of classifications $D = \langle c(x_1), \ldots, c(x_m) \rangle$
- Likelihood: $P(D \mid h) = \begin{cases} 1 & \text{if } h \text{ is consistent with } D \\ 0 & \text{otherwise} \end{cases}$
- Prior: uniform, $P(h) = \frac{1}{|H|}$ for all $h \in H$

Relation to Concept Learning: Posterior

First compute $P(D)$ with the theorem of total probability:

$$P(D) = \sum_{h_i \in H} P(D \mid h_i) P(h_i) = \sum_{h_i \in VS_{H,D}} 1 \cdot \frac{1}{|H|} = \frac{|VS_{H,D}|}{|H|}.$$

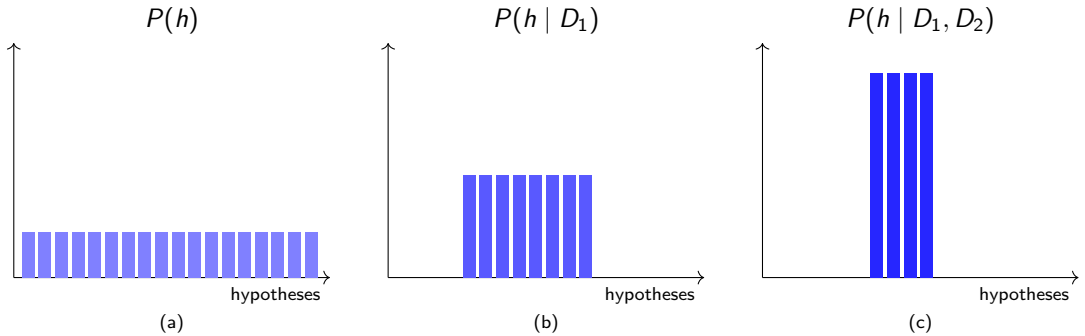
Hence, for a consistent h:

$$P(h \mid D) = \frac{1 \cdot \frac{1}{|H|}}{\frac{|VS_{H,D}|}{|H|}} = \frac{1}{|VS_{H,D}|}.$$

Result

$$P(h \mid D) = \begin{cases} \frac{1}{|VS_{H,D}|} & \text{if } h \text{ is consistent with } D \\ 0 & \text{otherwise} \end{cases}$$

Evolution of Posterior Probabilities (cf. Mitchell, Fig. 6.1)



- Initially every hypothesis has the same probability.
- As data arrive, inconsistent hypotheses drop to zero probability.
- The total mass ($= 1$) is shared equally among the remaining consistent hypotheses (the version space).

Characterizing Learners by Equivalent MAP Learners

- Under the assumptions above, **every hypothesis consistent with D is a MAP hypothesis**.
- A **consistent learner** is a learner that outputs only hypotheses that are consistent with the training data.

Theorem

IF

- the probability distribution on H is uniform, i.e. $P(h) = c$, $\forall h \in H$,
- D is noise-free (no conflicting data),
- D is deterministic: $P(D \mid h) = 1$ if D and h are consistent, else 0,

THEN **every consistent** learner outputs a MAP hypothesis.

Example 2: FIND-S Outputs a MAP Hypothesis

- FIND-S outputs a MAP hypothesis *even though it never explicitly manipulates probabilities*.
- The same holds for other choices of $P(h)$: any distribution that **favors more specific hypotheses**,

$$P(h_1) \geq P(h_2) \quad \text{if } h_1 \text{ is more specific than } h_2.$$

Why this matters

By identifying $P(D \mid h)$ and $P(h)$ under which a learner outputs the MAP hypothesis, the Bayesian framework **characterizes the behavior of learning algorithms** — even when no explicit use is made of these probabilities.

For FIND-S and CANDIDATE-ELIMINATION we use probabilities with values 0 or 1 only, capturing the deterministic aspect and noise-free data.

Outline

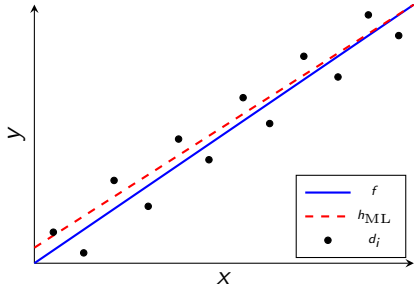
1. Bayes Theorem and MAP/ML Hypotheses
2. MAP Learners and Concept Learning
3. ML Hypotheses for Real-Valued and Probabilistic Targets
4. Minimum Description Length Principle
5. Bayes Optimal and Gibbs Classifiers
6. Naive Bayes Classifier
7. Learning to Classify Text
8. Bayesian Belief Networks
9. Expectation Maximization

Learning a Real-Valued Function

Consider any real-valued target function f. Under certain conditions, the ML hypothesis coincides with the **least squared error** hypothesis.

Training examples: $\langle x_i, d_i \rangle$ where

- $d_i = f(x_i) + e_i$ is a noisy training value
- e_i is noise drawn independently for each x_i from a Gaussian with mean 0



Proposition 1

Under these assumptions the maximum likelihood hypothesis minimizes the sum of squared errors:

$$h_{\mathrm{ML}} = \arg\min_{h \in H} \sum_{i=1}^{m} \big(d_i - h(x_i)\big)^2$$

Proof of Proposition 1

Examples are independent, and each $d_i \sim \mathcal{N}(h(x_i), \sigma^2)$ given h:

$$h_{\mathrm{ML}} = \arg\max_{h \in H} p(D \mid h) = \arg\max_{h \in H} \prod_{i=1}^{m} p(d_i \mid h)$$
$$= \arg\max_{h \in H} \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2}\left(\frac{d_i - h(x_i)}{\sigma}\right)^2}$$

Maximize the (monotone) natural logarithm instead:

$$h_{\mathrm{ML}} = \arg\max_{h \in H} \sum_{i=1}^{m} \left[\ln \frac{1}{\sqrt{2\pi\sigma^2}} - \frac{1}{2}\left(\frac{d_i - h(x_i)}{\sigma}\right)^2 \right] = \arg\max_{h \in H} \sum_{i=1}^{m} -\frac{1}{2}\left(\frac{d_i - h(x_i)}{\sigma}\right)^2$$
$$= \arg\max_{h \in H} \sum_{i=1}^{m} -\big(d_i - h(x_i)\big)^2 = \arg\min_{h \in H} \sum_{i=1}^{m} \big(d_i - h(x_i)\big)^2 \qquad \blacksquare$$

Learning to Predict Probabilities: Setup

We want to learn a function $f : X \to \{0, 1\}$, and we let

$$p_0(x) = P(f(x) = 0), \qquad p_1(x) = P(f(x) = 1) = 1 - p_0(x).$$

Example

$X = \{x : x \text{ is a patient with given symptoms}\}$ and $f(x) = \begin{cases} 1 & \text{if } x \text{ survives} \\ 0 & \text{otherwise} \end{cases}$

- Goal: learn $p(x) = P(f(x) = 1)$ — a *probability*, not a 0/1 label.
- Training examples $\langle x_i, d_i \rangle$ where $d_i \in \{0, 1\}$.
- For each x, $F_1(x)$ and $F_0(x)$ denote the frequencies of 1s and 0s observed.

State it as an ML problem: what is $P(D \mid h)$ for $D = \{(x_1, d_1), \ldots, (x_m, d_m)\}$?

Deriving the Likelihood $P(D \mid h)$

Claim: $P(x, d \mid h) = P(d \mid h, x)\, P(x \mid h)$.

$$\text{rhs} = \frac{P(d, h, x)}{P(h, x)} \cdot \frac{P(x, h)}{P(h)} = \frac{P(x, h, d)}{P(h)} = \text{lhs}.$$

Since x is independent of h, $P(x \mid h) = P(x)$. Applying this to every example:

$$P(D \mid h) = \prod_{i=1}^{m} P(x_i, d_i \mid h) = \prod_{i=1}^{m} P(d_i \mid h, x_i)\, P(x_i).$$

Because $h(x_i)$ is the predicted probability that $d_i = 1$:

$$P(d_i \mid h, x_i) = \begin{cases} h(x_i) & d_i = 1 \\ 1 - h(x_i) & d_i = 0 \end{cases} = h(x_i)^{d_i} \left(1 - h(x_i)\right)^{1 - d_i}$$

$$\Rightarrow \quad P(D \mid h) = \prod_{i=1}^{m} h(x_i)^{d_i} \left(1 - h(x_i)\right)^{1 - d_i} P(x_i).$$

ML Hypothesis: Cross-Entropy and Gradient Ascent

$P(x_i)$ does not depend on h; taking logarithms:

Maximum likelihood = minimum cross-entropy

$$h_{\mathrm{ML}} = \arg\max_{h \in H} \sum_{i=1}^{m} d_i \ln h(x_i) + (1 - d_i) \ln\bigl(1 - h(x_i)\bigr)$$

- The negative of this sum is the **cross-entropy** between the observed labels and the predictions.
- For a sigmoid unit (single-layer network), gradient ascent on this quantity gives:

Weight update rule

$$w_{jk} \leftarrow w_{jk} + \Delta w_{jk}, \qquad \Delta w_{jk} = \eta \sum_{i=1}^{m} \bigl(d_i - h(x_i)\bigr)\, x_{ijk}$$

Compare with backpropagation for squared error: the factor $h(1 - h)$ disappears.

Outline

1. Bayes Theorem and MAP/ML Hypotheses
2. MAP Learners and Concept Learning
3. ML Hypotheses for Real-Valued and Probabilistic Targets
4. Minimum Description Length Principle
5. Bayes Optimal and Gibbs Classifiers
6. Naive Bayes Classifier
7. Learning to Classify Text
8. Bayesian Belief Networks
9. Expectation Maximization

Minimum Description Length (MDL) Principle

Occam's razor: prefer the shortest hypothesis.

MDL: prefer the hypothesis h that minimizes

$$h_{\mathrm{MDL}} = \arg\min_{h \in H} \; L_{C_1}(h) + L_{C_2}(D \mid h)$$

where $L_C(x)$ is the description length of x under encoding C.

Example 3: H = decision trees, D = training data labels

- $L_{C_1}(h)$ = number of bits to describe tree h
- $L_{C_2}(D \mid h)$ = number of bits to describe D given h
  - $L_{C_2}(D \mid h) = 0$ if examples are classified perfectly by h
  - otherwise we need only describe the *exceptions*
- Hence h_{MDL} **trades off tree size for training errors**.

From MAP to MDL

Rewrite the MAP hypothesis using logarithms:

$$\begin{aligned} h_{\mathrm{MAP}} &= \arg\max_{h \in H} P(D \mid h)\, P(h) \\ &= \arg\max_{h \in H} \log_2 P(D \mid h) + \log_2 P(h) \\ &= \arg\min_{h \in H} -\log_2 P(D \mid h) - \log_2 P(h) \end{aligned} \tag{4}$$

Fact from information theory

The optimal (shortest expected coding length) code for an event with probability p uses $-\log_2 p$ bits.

So interpret (4):

1. $-\log_2 P(h)$ is the length of h under the optimal code for H
2. $-\log_2 P(D \mid h)$ is the length of D given h under the optimal code

$\Rightarrow$ prefer the hypothesis minimizing length(h) + length(misclassifications).

MDL: Interpretation and Caveats

- If C_1 and C_2 are the **optimal** codes for $P(h)$ and $P(D \mid h)$, then

$$h_{\mathrm{MDL}} = h_{\mathrm{MAP}}.$$

- MDL thus gives a Bayesian justification of Occam's razor: short hypotheses are those with high prior probability.
- In practice we rarely know the true priors, so we pick *some* encoding — the choice of encoding is equivalent to choosing a prior.

Trade-off

- A large tree can classify every example correctly: small L_{C_2}, large L_{C_1}.
- A small tree misclassifies some examples: small L_{C_1}, larger L_{C_2}.
- MDL picks the balance — a principled guard against overfitting.

Outline

1. Bayes Theorem and MAP/ML Hypotheses
2. MAP Learners and Concept Learning
3. ML Hypotheses for Real-Valued and Probabilistic Targets
4. Minimum Description Length Principle
5. Bayes Optimal and Gibbs Classifiers
6. Naive Bayes Classifier
7. Learning to Classify Text
8. Bayesian Belief Networks
9. Expectation Maximization

Most Probable Classification of New Instances

So far we have sought the most probable *hypothesis* given the data (h_{MAP}).

Given a new instance x, what is its most probable *classification*?

$h_{\mathrm{MAP}}(x)$ is **not** necessarily the most probable classification!

Consider three possible hypotheses:

h	$P(h \mid D)$	$h(x)$
h_1	0.4	$+$
h_2	0.3	$-$
h_3	0.3	$-$

$h_{\mathrm{MAP}} = h_1$ predicts $+$, but the total posterior mass predicting $-$ is 0.6.

Bayes Optimal Classifier

Bayes optimal classification

$$\arg\max_{v \in V} \sum_{h_i \in H} P(v \mid h_i)\, P(h_i \mid D)$$

Example:

$$\begin{aligned} P(h_1 \mid D) &= 0.4, & P(- \mid h_1) &= 0, & P(+ \mid h_1) &= 1 \\ P(h_2 \mid D) &= 0.3, & P(- \mid h_2) &= 1, & P(+ \mid h_2) &= 0 \\ P(h_3 \mid D) &= 0.3, & P(- \mid h_3) &= 1, & P(+ \mid h_3) &= 0 \end{aligned}$$

therefore

$$\sum_{h_i} P(+ \mid h_i) P(h_i \mid D) = 0.4, \qquad \sum_{h_i} P(- \mid h_i) P(h_i \mid D) = 0.6$$

$$\Rightarrow \quad \arg\max_{v \in V} \sum_{h_i \in H} P(v \mid h_i)\, P(h_i \mid D) = -$$

No other classifier using the same H and prior can outperform it on average.

Gibbs Classifier

The Bayes optimal classifier gives the best result, but it can be **expensive** when there are many hypotheses.

Gibbs algorithm

1. Choose one hypothesis at random, according to $P(h \mid D)$
2. Use it to classify the new instance

Surprising fact

If target concepts are drawn at random from H according to the priors on H, then

$$E[\text{error}_{\text{Gibbs}}] \leq 2\, E[\text{error}_{\text{BayesOptimal}}]$$

With a correct, uniform prior over H:

- pick *any* hypothesis from the version space, uniformly at random
- its expected error is no worse than twice the Bayes optimal error

Outline

1. Bayes Theorem and MAP/ML Hypotheses
2. MAP Learners and Concept Learning
3. ML Hypotheses for Real-Valued and Probabilistic Targets
4. Minimum Description Length Principle
5. Bayes Optimal and Gibbs Classifiers
6. Naive Bayes Classifier
7. Learning to Classify Text
8. Bayesian Belief Networks
9. Expectation Maximization

Naive Bayes Classifier: When to Use

Along with decision trees, neural networks and nearest neighbor, one of the **most practical** learning methods.

When to use

- Moderate or large training set available
- Attributes describing instances are *conditionally independent given the classification*

Successful applications

- Diagnosis
- Classifying text documents

Setting: target function $f : X \to V$; each instance x is described by attributes $\langle a_1, a_2, \ldots, a_n \rangle$.

Naive Bayes: Derivation

Most probable value of $f(x)$:

$$\begin{aligned} v_{\mathrm{MAP}} &= \arg\max_{v \in V} P(v \mid a_1, a_2, \ldots, a_n) \\ &= \arg\max_{v \in V} \frac{P(a_1, a_2, \ldots, a_n \mid v)\, P(v)}{P(a_1, a_2, \ldots, a_n)} = \arg\max_{v \in V} P(a_1, a_2, \ldots, a_n \mid v)\, P(v) \end{aligned}$$

Problem: estimating $P(a_1, \ldots, a_n \mid v)$ needs a number of parameters exponential in n.

Naive Bayes assumption

$$P(a_1, a_2, \ldots, a_n \mid v) = \prod_{i=1}^{n} P(a_i \mid v)$$

Naive Bayes classifier

$$v_{\mathrm{NB}} = \arg\max_{v \in V} P(v) \prod_{i=1}^{n} P(a_i \mid v)$$

Naive Bayes: Algorithm

NAIVE_BAYES_LEARN(*examples*)

For each target value v

- $\widehat{P}(v) \leftarrow$ estimate $P(v)$
- For each attribute value a_i of each attribute a: $\widehat{P}(a_i \mid v) \leftarrow$ estimate $P(a_i \mid v)$

CLASSIFY_NEW_INSTANCE(x)

$$v_{\mathrm{NB}} = \arg\max_{v \in V} \widehat{P}(v) \prod_{a_i \in x} \widehat{P}(a_i \mid v)$$

- Estimates are simple frequency counts over the training data.
- No search through a hypothesis space — learning is a single pass.
- Number of parameters: $|V| \cdot (1 + \sum_a |\mathrm{values}(a)|)$, linear in n.

Naive Bayes Example: *PlayTennis* Data

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

$V = \{y, n\}$; A is the set of attributes, $a \in A$ an attribute, a_i the i-th value of a.

Naive Bayes Example: Classifying a New Instance

New instance: $\langle Outlook = \text{sun},\ Temp = \text{cool},\ Humid = \text{high},\ Wind = \text{strong} \rangle$

$$v_{\text{NB}} = \arg\max_{v \in V} P(v) \prod_{a \in A} P(a_i \mid v)$$

	$v = y$	$v = n$
$P(v)$	9/14	5/14
$P(\text{sun} \mid v)$	2/9	3/5
$P(\text{cool} \mid v)$	3/9	1/5
$P(\text{high} \mid v)$	3/9	4/5
$P(\text{strong} \mid v)$	3/9	3/5
Product	**0.005**	**0.021**

$\Rightarrow\ v_{\text{NB}} = n.$ Normalized: $P(n \mid x) \approx \dfrac{0.021}{0.021 + 0.005} \approx 0.795.$

Naive Bayes Subtleties (1): Independence Is Often Violated

$$P(a_1, a_2, \ldots, a_n \mid v) = \prod_i P(a_i \mid v)$$

- The conditional independence assumption is **often violated** in practice...
- ...but Naive Bayes works **surprisingly well** anyway.
- We don't need the estimated posteriors $\widehat{P}(v \mid x)$ to be correct; we only need

$$\arg\max_{v \in V} \widehat{P}(v) \prod_i \widehat{P}(a_i \mid v) \;=\; \arg\max_{v \in V} P(v)\, P(a_1, \ldots, a_n \mid v)$$

- See Domingos & Pazzani (1996) for an analysis.
- Naive Bayes posteriors are often **unrealistically close to** 1 **or** 0.

Naive Bayes Subtleties (2): Zero Counts and the m-Estimate

What if no training instance with target value v has attribute value a_i? Then

$$\widehat{P}(a_i \mid v) = 0 \quad \Rightarrow \quad \widehat{P}(v) \prod_i \widehat{P}(a_i \mid v) = 0$$

— a single unseen value wipes out all other evidence.

Typical solution: Bayesian (m-)estimate

$$\widehat{P}(a_i \mid v) \leftarrow \frac{n_c + m\,p}{n + m}$$

- n = number of training examples with target v
- n_c = number of examples with target v and $a = a_i$
- p = prior estimate for $\widehat{P}(a_i \mid v)$ (e.g. uniform: $p = 1/k$ for k values)
- m = weight given to the prior (number of "virtual" examples)

Outline

1. Bayes Theorem and MAP/ML Hypotheses
2. MAP Learners and Concept Learning
3. ML Hypotheses for Real-Valued and Probabilistic Targets
4. Minimum Description Length Principle
5. Bayes Optimal and Gibbs Classifiers
6. Naive Bayes Classifier
7. Learning to Classify Text
8. Bayesian Belief Networks
9. Expectation Maximization

Learning to Classify Text

Why?

- Learn which news articles are of interest
- Learn to classify web pages by topic

Naive Bayes is among the most effective algorithms for this task.

Target concept *Interesting?* : $Document \rightarrow \{+, -\}$

1. **Representation:** each document is a vector of words — one attribute per word position
2. **Learning:** use training examples to estimate $P(+)$, $P(-)$, $P(doc \mid +)$, $P(doc \mid -)$

Naive Bayes conditional independence assumption

$$P(doc \mid v) = \prod_{i=1}^{\text{length}(doc)} P(a_i = w_k \mid v)$$

$P(a_i = w_k \mid v)$: probability that the word in position i is w_k, given v.
One more assumption: $P(a_i = w_k \mid v) = P(a_m = w_k \mid v)$, $\forall i, m$ (position independence).

LEARN_NAIVE_BAYES_TEXT(*Examples*, V)

1. Collect all words and other tokens that occur in *Examples*
   - *Vocabulary* $\leftarrow$ all distinct words and tokens in *Examples*
2. Calculate the required $P(v)$ and $P(w_k \mid v)$ terms. For each target value $v \in V$:
   - $docs_v \leftarrow$ subset of *Examples* with target value v
   - $P(v) \leftarrow \dfrac{|docs_v|}{|Examples|}$
   - $Text_v \leftarrow$ a single document concatenating all members of $docs_v$
   - $n \leftarrow$ total number of words in $Text_v$ (counting duplicates)
   - For each word w_k in *Vocabulary*:
     - $n_k \leftarrow$ number of times w_k occurs in $Text_v$
     - $P(w_k \mid v) \leftarrow \dfrac{n_k + 1}{n + |Vocabulary|}$

Note: the estimate is an m-estimate with uniform prior $p = 1/|Vocabulary|$ and $m = |Vocabulary|$ (Laplace smoothing).

Classifying Text and the 20 Newsgroups Experiment

CLASSIFY_NAIVE_BAYES_TEXT(*Doc*)

- *positions* ← all word positions in *Doc* containing tokens found in *Vocabulary*
- Return $v_{\mathrm{NB}} = \arg\max_{v \in V} P(v) \prod_{i \in positions} P(a_i \mid v)$

Twenty Newsgroups: 1000 training documents from each group; classify new documents by source group.

comp.graphics
comp.os.ms-windows.misc
comp.sys.ibm.pc.hardware
comp.sys.mac.hardware
comp.windows.x
alt.atheism
soc.religion.christian
talk.religion.misc
talk.politics.mideast
talk.politics.misc
talk.politics.guns
misc.forsale
rec.autos
rec.motorcycles
rec.sport.baseball
rec.sport.hockey
sci.space
sci.crypt
sci.electronics
sci.med

Naive Bayes: **89% classification accuracy** (random guessing: 5%)

Outline

1. Bayes Theorem and MAP/ML Hypotheses
2. MAP Learners and Concept Learning
3. ML Hypotheses for Real-Valued and Probabilistic Targets
4. Minimum Description Length Principle
5. Bayes Optimal and Gibbs Classifiers
6. Naive Bayes Classifier
7. Learning to Classify Text
8. Bayesian Belief Networks
9. Expectation Maximization

Bayesian Belief Networks: Motivation

Also called **Bayes nets**. Interesting because:

- The Naive Bayes assumption of conditional independence is **too restrictive**
- But learning/inference is **intractable** without some such assumptions
- Bayesian belief networks describe conditional independence among *subsets* of variables

$\Rightarrow$ allows combining prior knowledge about (in)dependencies among variables with observed training data.

Conditional independence

X is conditionally independent of Y given Z if the probability distribution governing X is independent of the value of Y given the value of Z:

$$(\forall x_i, y_j, z_k)\ P(X = x_i \mid Y = y_j, Z = z_k) = P(X = x_i \mid Z = z_k)$$

or, compactly, $P(X \mid Y, Z) = P(X \mid Z)$.

Conditional Independence: Examples

Example 4

Thunder is conditionally independent of *Rain*, given *Lightning*:

$$P(\mathit{Thunder} \mid \mathit{Rain}, \mathit{Lightning}) = P(\mathit{Thunder} \mid \mathit{Lightning})$$

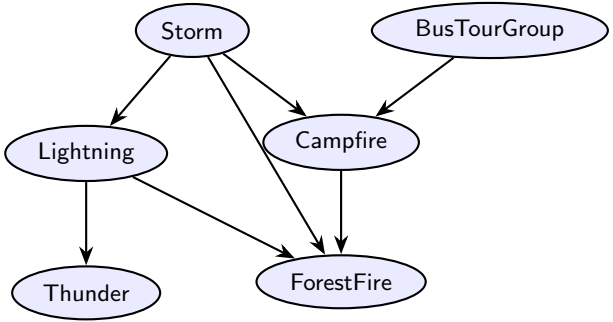
Once we know whether there was lightning, rain tells us nothing more about thunder.

Naive Bayes uses conditional independence to justify

$$\begin{aligned} P(X, Y \mid Z) &= P(X \mid Y, Z)\, P(Y \mid Z) && \text{(product rule)} \\ &= P(X \mid Z)\, P(Y \mid Z) && \text{(conditional independence)} \end{aligned}$$

which is very useful computationally: joint conditionals factor into one-variable terms.

A Bayesian Belief Network (Mitchell, Fig. 6.3)



CPT for *Campfire*

	S, B	$S, \neg B$	$\neg S, B$	$\neg S, \neg B$
C	0.4	0.1	0.8	0.2
$\neg C$	0.6	0.9	0.2	0.8

C = Campfire, S = Storm, B = BusTourGroup.

- Each node is asserted to be conditionally independent of its nondescendants, given its immediate predecessors (parents).
- The graph is a **directed acyclic graph**; each node carries a conditional probability table.

Bayes Nets Represent the Joint Distribution

A network represents the joint probability distribution over all its variables, e.g. $P(\mathit{Storm}, \mathit{BusTourGroup}, \ldots, \mathit{ForestFire})$.

Factorization

$$P(y_1, \ldots, y_n) = \prod_{i=1}^{n} P\big(y_i \mid \mathit{Parents}(Y_i)\big)$$

where $\mathit{Parents}(Y_i)$ denotes the immediate predecessors of Y_i in the graph.

For Figure 6.3:

$$P(S, B, L, C, T, F) = P(S)\, P(B)\, P(L \mid S)\, P(C \mid S, B)\, P(T \mid L)\, P(F \mid S, L, C)$$

- The joint distribution is fully defined by the graph plus the tables $P(y_i \mid \mathit{Parents}(Y_i))$.
- With 6 Boolean variables: $1 + 1 + 2 + 4 + 2 + 8 = 18$ parameters instead of $2^6 - 1 = 63$.

Inference in Bayesian Networks

Question: how can one infer the (probabilities of) values of one or more network variables, given observed values of others?

- The Bayes net contains all the information needed for this inference.
- If only one variable has an unknown value, it is easy to infer it.
- In the general case, the problem is **NP-hard**.

In practice, inference can succeed in many cases

- **Exact** inference methods work well for some network structures (e.g. polytrees).
- **Approximate** methods (e.g. Monte Carlo sampling) handle the rest.

Learning Bayesian Networks

Several variants of the learning task:

	All variables observed	**Some variables hidden**
Structure known	Easy: like training Naive Bayes	Gradient ascent / EM
Structure unknown	Structure search	Hardest: active research

Suppose the structure is known but some variables are unobserved:

- e.g. observe *ForestFire*, *Storm*, *BusTourGroup*, *Thunder*, but not *Lightning*, *Campfire*
- Similar to training a neural network with hidden units
- We can learn the conditional probability tables by **gradient ascent**: set up an objective function to be maximized
- Converge to a network h that (locally) maximizes $P(D \mid h)$

Gradient Ascent for Bayes Nets

Let w_{ijk} denote one entry in the conditional probability table for variable Y_i:

$$w_{ijk} = P\big(Y_i = y_{ij} \mid \mathit{Parents}(Y_i) = \text{the list } u_{ik} \text{ of values}\big)$$

e.g. if $Y_i = \mathit{Campfire}$, then u_{ik} might be $\langle \mathit{Storm} = T, \mathit{BusTourGroup} = F \rangle$.

Idea: to maximize a quantity, compute its gradient (vector of partial derivatives) and move the parameters in that direction.

Gradient ascent training (Russell et al.; Mitchell pp. 188–189)

1. Update all w_{ijk} using training data D:

$$w_{ijk} \leftarrow w_{ijk} + \eta \sum_{d \in D} \frac{P_h(y_{ij}, u_{ik} \mid d)}{w_{ijk}}$$

2. Renormalize the w_{ijk} so that $\sum_j w_{ijk} = 1$ and $0 \leq w_{ijk} \leq 1$.

Converges to a locally maximum likelihood hypothesis for the network's conditional probabilities.

Structure Learning and EM for Bayes Nets

Learning the structure — very difficult (ongoing research)

- Greedy search among network structures on a cost function that trades off structure complexity for accuracy (add/subtract edges and nodes)
- Constraint-based approaches: infer (in)dependencies from data, then build a consistent graph

EM can also be used for the CPTs. Repeatedly:

1. Calculate probabilities of unobserved variables, assuming the current h
2. Calculate new w_{ijk} to maximize $E[\ln P(D \mid h)]$, where D now includes both observed values and (calculated) probabilities of unobserved variables

Summary: Bayesian Belief Networks

- Combine prior knowledge with observed data
- Impact of prior knowledge (when correct!) is to **lower the sample complexity**
- Active research area:
  - Extend from Boolean to real-valued variables
  - Parameterized distributions instead of tables
  - Extend to first-order instead of propositional systems
  - More effective inference methods
  - ...

Outline

1. Bayes Theorem and MAP/ML Hypotheses
2. MAP Learners and Concept Learning
3. ML Hypotheses for Real-Valued and Probabilistic Targets
4. Minimum Description Length Principle
5. Bayes Optimal and Gibbs Classifiers
6. Naive Bayes Classifier
7. Learning to Classify Text
8. Bayesian Belief Networks
9. Expectation Maximization

Expectation Maximization (EM): When to Use

When to use

- Data is only **partially observable**
- Unsupervised clustering (target value unobservable)
- Supervised learning (some instance attributes unobservable)

Some uses

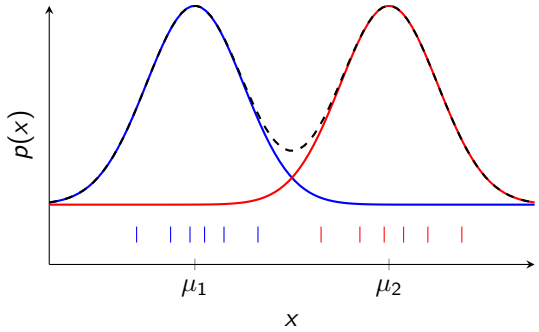
- Train Bayesian belief networks
- Unsupervised clustering (AUTOCLASS)
- Learning Hidden Markov Models

Generating Data from a Mixture of k Gaussians

Each instance x is generated by

1. choosing one of the k Gaussians with uniform probability
2. generating an instance at random according to that Gaussian

We see the points x_i — but **not** which Gaussian produced each one.



EM for Estimating k Means: Problem

Given

- Instances from X generated by a mixture of k Gaussian distributions
- Unknown means $\langle \mu_1, \ldots, \mu_k \rangle$ of the k Gaussians
- We don't know which instance x_i was generated by which Gaussian

Determine

Maximum likelihood estimates of $\langle \mu_1, \ldots, \mu_k \rangle$

Full description of each instance ($k = 2$): $y_i = \langle x_i, z_{i1}, z_{i2} \rangle$, where

- $z_{ij} = 1$ if x_i was generated by the j-th Gaussian, else 0
- x_i is observable
- z_{ij} is **unobservable** (hidden)

EM for Estimating k Means: Algorithm

Pick a random initial $h = \langle \mu_1, \mu_2 \rangle$, then iterate:

E step

Calculate the expected value $E[z_{ij}]$ of each hidden variable, assuming $h = \langle \mu_1, \mu_2 \rangle$ holds:

$$E[z_{ij}] = \frac{p(x = x_i \mid \mu = \mu_j)}{\sum_{n=1}^{2} p(x = x_i \mid \mu = \mu_n)} = \frac{e^{-\frac{1}{2\sigma^2}(x_i - \mu_j)^2}}{\sum_{n=1}^{2} e^{-\frac{1}{2\sigma^2}(x_i - \mu_n)^2}}$$

M step

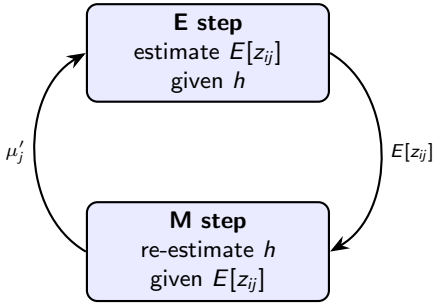
Calculate a new ML hypothesis $h' = \langle \mu'_1, \mu'_2 \rangle$, assuming each z_{ij} takes its expected value:

$$\mu_j \leftarrow \frac{\sum_{i=1}^{m} E[z_{ij}]\, x_i}{\sum_{i=1}^{m} E[z_{ij}]}$$

Note the similarity to the sample mean of a single distribution — each point is weighted by its "responsibility".

The Essence of EM

- The **current hypothesis** is used to estimate the unobserved variables.
- The **expected values** of these variables are then used to calculate an improved hypothesis.
- On each iteration, EM increases the likelihood $P(D \mid h)$ unless it is at a local maximum.
- Hence EM converges to a **local** maximum likelihood hypothesis for $\langle \mu_1, \mu_2 \rangle$.



Practical note

Since only a local maximum is guaranteed, the result depends on initialization; running EM from several random starts is common.

Summary

- **Bayes theorem** combines prior knowledge with observed data: $P(h \mid D) \propto P(D \mid h)P(h)$.
- **MAP / ML hypotheses**: consistent learners are MAP learners under uniform priors and noise-free data; least squares is ML under Gaussian noise; cross-entropy is ML for probabilistic targets.
- **MDL** recasts MAP as minimizing $\text{length}(h) + \text{length}(D \mid h)$ — a Bayesian Occam's razor.
- **Bayes optimal classifier** combines all hypotheses weighted by posterior; **Gibbs** samples one and is within a factor of 2.
- **Naive Bayes**: simple, fast, effective (e.g. 89% on 20 Newsgroups) despite its strong independence assumption.
- **Bayesian belief networks** encode conditional independence among subsets of variables; CPTs can be learned by gradient ascent or EM.
- **EM** handles hidden variables by alternating expectation and maximization, converging to a local ML hypothesis.